

FLAWFIX SECUREASSIST

Automated Vulnerability Detection, Semantic LLVM IR Analysis & Patch Verification Audit Report

Target File: samplerust.rs_patched	Language: RUST
Project: FlawFix Workspace	Audit Date: 2026-09-26 11:52:15
Report ID: RPT-DEF5D61A	Engine: FlawFix LLVM+XAI v1.0

1. Executive Security Assessment

The provided Rust code is memory-safe and robust. It correctly handles user input by parsing it into a usize, enforcing a strict upper bound (10,000,000) to prevent excessive memory allocation (Denial of Service), and uses Rust's standard library which manages memory automatically, preventing buffer overflows and use-after-free vulnerabilities.

TOTAL FLAWS	CRITICAL	HIGH	MEDIUM	LOW
0	0	0	0	0

2. Detected Vulnerability Inventory

✓ No security vulnerabilities detected in this source code.

4. Automated Patch Verification Audit

Patch for VULN-001: **VERIFIED & COMPILER-VALIDATED**

The original code was vulnerable to an Uncontrolled Resource Consumption (Denial of Service) via memory exhaustion, as it allowed an attacker to specify an arbitrary size for a Vec allocation. The patch introduces a constant MAX_DATA_SIZE and uses std::cmp::min to clamp the user-provided input, effectively preventing excessive memory allocation. The use of unwrap_or(0) gracefully handles parsing errors or empty inputs, preventing panics.

Patch for VULN-001: **VERIFIED & COMPILER-VALIDATED**

The original code was vulnerable to an Uncontrolled Resource Consumption (Denial of Service) attack, as it allowed an attacker to provide an arbitrarily large integer to Vec::with_capacity, potentially leading to an out-of-memory (OOM) crash. The patch introduces a strict upper bound (10,000,000) on the input size, effectively mitigating the risk of excessive memory allocation. The use of a match statement with a guard is idiomatic Rust and correctly handles both parsing errors and out-of-bounds values.

5. AI Code Optimization Recommendations

- Use of try_reserve for memory safety:** While the current code is safe, using Vec::try_reserve instead of Vec::with_capacity allows for graceful handling of allocation failures if the system runs out of memory. (Impact: Improves robustness in memory-constrained environments.)
- Input validation refinement:** The current input parsing is sufficient, but using a more specific error handling pattern (e.g., Result mapping) could improve code readability. (Impact: Better maintainability and clearer error reporting.)